Japanese Language Learning Web App — Implementation Plan

Context

Building a full-stack Japanese language learning web app that integrates with Spotify to show annotated lyrics (furigana + English translation) as a companion page, plus a personal vocabulary dictionary and flashcard quiz system. The user wants to learn Japanese using real music lyrics as a study source. Stack: Next.js 15, Supabase, OpenRouter (Gemma 3 27B free), Genius API, Spotify Web API.

Tech Stack

Concern	Choice
Framework	Next.js 15 (App Router, TypeScript)
Styling	Tailwind CSS
Database + Auth	Supabase (Postgres + Supabase Auth)
AI	OpenRouter → google/gemma-3-27b-it:free
Lyrics (primary)	lrclib.net — free, no API key, synced LRC format
Lyrics (fallback)	Genius API — search + server-side HTML scraping
Spotify	Spotify Web API (OAuth + currently-playing with `progress_ms`)

Features

1. **Lyrics Companion (`/lyrics`)** — detect currently playing Spotify song, fetch synced LRC lyrics (lrclib.net primary, Genius fallback), generate furigana + line-by-line English translation via AI, display with `<ruby>` annotations and **karaoke-style real-time line highlighting** using Spotify `progress_ms`. User can select a phrase to save to dictionary.
2. **Dictionary (`/dictionary`)** — CRUD vocabulary list. Manual add triggers AI to populate hiragana, translation, and example sentence. Saved entries include optional song source context. Sorting by date/Japanese/English + tag filter + search.
3. **Flashcards (`/flashcards`)** — configurable quiz sessions from dictionary entries. JP→EN or EN→JP mode. Flip animation, got-it/missed tracking, session summary.
4. **Auth** — Supabase Auth (email + Google OAuth). All data is per-user with Supabase RLS.

Edge Case Handling

- **Non-Japanese song**: Detect Japanese characters (kanji U+4E00–U+9FFF, hiragana U+3040–U+309F, katakana U+30A0–U+30FF). If none found → show "No Japanese lyrics detected" notice, hide furigana/translation panels.
- **Romaji-only lyrics**: If lyrics are detected as romaji (Latin script only), send to AI for **romaji → proper Japanese conversion** first (romaji → hiragana is deterministic; hiragana → kanji uses AI context). Then run furigana + translation on the converted Japanese.
- **Lyrics not found**: Show "Lyrics not found on lrclib.net or Genius" with a **manual paste text area**. User-pasted lyrics are processed the same way (furigana + translation). No karaoke sync for manually pasted lyrics.
- **Instrumental / no lyrics**: Handle 404/empty responses from both lyrics sources gracefully.

File Structure

```
c:\Users\User\Journey\claude-test\
├── .env.local
├── next.config.ts
├── tailwind.config.ts
├── supabase/
│   └── schema.sql
└── src/
    ├── middleware.ts              # Session refresh + route protection
    ├── app/
    │   ├── layout.tsx
    │   ├── page.tsx               # Landing → redirect
```

```
│   ├── auth/
│   │   ├── login/page.tsx
│   │   ├── signup/page.tsx
│   │   ├── callback/route.ts   # Supabase OAuth callback
│   │   └── logout/route.ts
│   ├── lyrics/page.tsx
│   ├── dictionary/page.tsx
│   ├── flashcards/page.tsx
│   └── api/
│       ├── spotify/connect/route.ts
│       ├── spotify/callback/route.ts
│       ├── spotify/now-playing/route.ts
│       ├── spotify/refresh/route.ts
│       ├── lyrics/route.ts             # Unified: lrclib → Genius fallback
│       ├── ai/furigana/route.ts
│       ├── ai/translate/route.ts
│       ├── ai/enrich-word/route.ts
│       ├── ai/romaji-to-japanese/route.ts
│       ├── dictionary/route.ts
│       └── dictionary/[id]/route.ts
├── components/
│   ├── layout/Navbar.tsx
│   ├── lyrics/
│   │   ├── NowPlayingBanner.tsx
│   │   ├── LyricsDisplay.tsx   # Karaoke sync: highlights active line via progress_ms
│   │   ├── LyricsLine.tsx      # Active line gets highlighted style + auto-scroll
│   │   ├── RubyText.tsx        # Renders <ruby><rt> from token array
│   │   ├── ManualLyricsInput.tsx  # Shown when lyrics not found
│   │   ├── PhraseSelector.tsx  # selectionchange listener → save popover
│   │   └── SaveToDictionaryModal.tsx
│   ├── dictionary/
│   │   ├── DictionaryTable.tsx
│   │   ├── DictionaryEntry.tsx
│   │   ├── AddWordModal.tsx
│   │   ├── EditWordModal.tsx
│   │   ├── SortControls.tsx
│   │   └── TagFilter.tsx
│   └── flashcards/
│       ├── FlashcardSetup.tsx
│       ├── FlashcardSession.tsx
│       ├── FlashcardCard.tsx   # CSS 3D flip animation
│       ├── ModeSelector.tsx
│       └── SessionSummary.tsx
├── lib/
│   ├── supabase/client.ts      # Browser singleton
│   ├── supabase/server.ts      # Server client (cookies)
│   ├── spotify/api.ts          # getNowPlaying(), refreshToken()
│   ├── spotify/auth.ts         # OAuth URL builder
│   ├── lrclib/client.ts        # getLyrics(track, artist) → LrcLine[]
│   ├── genius/scraper.ts       # searchSong() + scrapeLyrics()
│   ├── openrouter/client.ts    # Base fetch wrapper
│   ├── openrouter/furigana.ts  # Prompt + JSON parser
│   ├── openrouter/translate.ts
│   ├── openrouter/enrich-word.ts
│   ├── openrouter/romaji-to-japanese.ts  # Romaji → kanji/kana conversion
│   ├── utils/japanese.ts       # hasJapaneseChars(), isRomaji()
│   ├── utils/lrc-parser.ts     # Parse LRC → { ms: number, text: string }[]
│   └── utils/lyrics-parser.ts  # Clean scraped Genius HTML → string[]
├── hooks/
│   ├── useNowPlaying.ts        # Polls every 3s (returns progress_ms too)
│   ├── useLyrics.ts            # Cached per track+artist, returns LrcLine[]
```

```
│   ├── useFurigana.ts          # Batched AI requests
│   └── useDictionary.ts        # CRUD operations
└── types/
    ├── database.ts
    ├── spotify.ts
    └── ai.ts                   # FuriganaToken, TranslatedLine
```

Supabase Schema

```sql
create extension if not exists "uuid-ossp";

-- Spotify OAuth tokens (one row per user)
create table public.spotify_tokens (
  id            uuid primary key default uuid_generate_v4(),
  user_id       uuid not null references auth.users(id) on delete cascade unique,
  access_token  text not null,
  refresh_token text not null,
  expires_at    timestamptz not null,
  created_at    timestamptz not null default now(),
  updated_at    timestamptz not null default now()
);

-- Vocabulary dictionary
create table public.dictionary_entries (
  id                  uuid primary key default uuid_generate_v4(),
  user_id             uuid not null references auth.users(id) on delete cascade,
  japanese_text       text not null,
  hiragana            text not null,
  english_translation text not null,
  example_japanese    text,
  example_english     text,
  source_song         text,
  source_artist       text,
  source_lyrics_line  text,
  tags                text[] not null default '{}',
  created_at          timestamptz not null default now(),
  updated_at          timestamptz not null default now()
);

create index idx_dict_user_id    on public.dictionary_entries(user_id);
create index idx_dict_created_at on public.dictionary_entries(created_at desc);
create index idx_dict_jp_text    on public.dictionary_entries(japanese_text);
create index idx_dict_tags       on public.dictionary_entries using gin(tags);

-- RLS
alter table public.spotify_tokens     enable row level security;
alter table public.dictionary_entries enable row level security;

-- spotify_tokens: full CRUD for own row
create policy "own spotify tokens" on public.spotify_tokens
  using (auth.uid() = user_id) with check (auth.uid() = user_id);

-- dictionary_entries: full CRUD for own rows
create policy "own dictionary entries" on public.dictionary_entries
  using (auth.uid() = user_id) with check (auth.uid() = user_id);

-- updated_at trigger
create or replace function public.set_updated_at()
returns trigger language plpgsql as $$
```

```
begin new.updated_at = now(); return new; end; $$;

create trigger trg_spotify_tokens_upd     before update on public.spotify_tokens     for each row execute function
public.set_updated_at();
create trigger trg_dictionary_entries_upd before update on public.dictionary_entries for each row execute function
public.set_updated_at();
```

Key API Design Notes

Lyrics Fetching (priority chain)

`GET /api/lyrics?track=...&artist=...` — unified route:

1. Try **lrclib.net** first: `GET https://lrclib.net/api/get?track_name={track}&artist_name={artist}`
   - Returns JSON with `syncedLyrics` (LRC format) or `plainLyrics`
   - Parse LRC timestamps (`[mm:ss.xx]`) → `{ ms: number, text: string }[]`
   - Return `{ lines: LrcLine[], synced: true }`
2. If lrclib returns 404 or empty, fall back to **Genius**:
   - Search Genius API, scrape song page, parse `data-lyrics-container="true"` divs
   - Return `{ lines: LrcLine[], synced: false }` (no timestamps)
3. If both fail → return `{ lines: [], synced: false, notFound: true }` (client shows manual paste UI)

Japanese detection + romaji conversion (server-side, before AI):

- `hasJapaneseChars(text)` : check for kanji U+4E00–U+9FFF, hiragana U+3040–U+309F, katakana U+30A0–U+30FF
- If **no Japanese chars found** → return `{ isJapanese: false }` (client shows "No Japanese lyrics" notice)
- If **romaji detected** (all ASCII, no Japanese chars): call `/api/ai/romaji-to-japanese` first
  - AI prompt: convert romanized Japanese lyrics to native Japanese (kanji + kana)
  - Then run furigana + translation on the converted result

Genius Lyrics Scraping (fallback only)

1. Call `GET https://api.genius.com/search?q={track} {artist}` with Bearer token
2. Match best result by title + artist name
3. Server-fetch the Genius song page HTML with a realistic `User-Agent` header
4. Parse `div[data-lyrics-container="true"]` elements using `node-html-parser` ; convert `<br>` to newlines, strip annotation `<a>` tags
5. Return plain lines (no timestamps)

AI Furigana (`/api/ai/furigana`)

- Combined single OpenRouter call returns both furigana tokens AND line translations to halve API calls
- Response format: `{ furigana: Array<Array<{original: string, reading: string|null}>>, translations: string[] }`
- Process lyrics in batches of 25 lines to stay within token limits
- Use `response_format: { type: "json_object" }` + `temperature: 0.1`
- Strip markdown fences before `JSON.parse()` as fallback

AI Word Enrichment (`/api/ai/enrich-word`)

- Returns `{ hiragana, english_translation, example_japanese, example_english }`
- Triggered on blur of the Japanese input field in AddWordModal / SaveToDictionaryModal

Spotify Token Refresh

- Check `expires_at > now + 5min` before each `/api/spotify/now-playing` call
- Refresh inline if needed, update DB, then proceed
- Store `NEXT_PUBLIC_APP_URL` in env to build the redirect URI dynamically

Dictionary Sorting (`GET /api/dictionary`)

`?sort=` value	SQL
`created_at_desc` (default)	`order by created_at desc`

`created_at_asc`	`order by created_at asc`
`japanese_asc`	`order by japanese_text asc`
`english_asc`	`order by english_translation asc`

Implementation Order

Phase 1 — Scaffolding

1. `npx create-next-app@latest` with TypeScript + Tailwind + App Router
2. Install: `@supabase/supabase-js @supabase/ssr node-html-parser lucide-react`
3. Set up `.env.local`, `supabase/schema.sql`
4. Create `src/lib/supabase/client.ts` + `server.ts`
5. Create `src/middleware.ts` (session refresh + protect `/lyrics`, `/dictionary`, `/flashcards`)

Phase 2 — Auth

6. Login / signup pages + Supabase Auth
7. `src/app/auth/callback/route.ts` (code exchange)
8. Navbar with logout

Phase 3 — Dictionary Core

9. `/api/dictionary` GET + POST routes
10. `/api/dictionary/[id]` PATCH + DELETE
11. Dictionary page components + `useDictionary` hook
12. Test CRUD + verify RLS

Phase 4 — AI Word Enrichment

13. `src/lib/openrouter/client.ts` + `enrich-word.ts`
14. `/api/ai/enrich-word` route
15. Wire into AddWordModal (AI enrichment on blur)

Phase 5 — Spotify Integration

16. `/api/spotify/connect` + `/api/spotify/callback` (OAuth + token storage)
17. `/api/spotify/now-playing` (read tokens, refresh if needed, call Spotify)
18. `useNowPlaying` hook (3s polling with `setInterval` + cleanup)
19. `SpotifyConnectButton`, `NowPlayingBanner` components

Phase 6 — Lyrics Fetching (lrclib + Genius fallback)

20. `src/lib/lrclib/client.ts` — `getLyrics(track, artist)` → `{ syncedLyrics, plainLyrics }`
21. `src/lib/utils/lrc-parser.ts` — parse LRC timestamps → `{ ms, text }[]`
22. `src/lib/genius/scraper.ts` — `searchSong()` + `scrapeLyrics()`
23. `src/lib/utils/lyrics-parser.ts` + `src/lib/utils/japanese.ts` (hasJapaneseChars, isRomaji)
24. Unified `/api/lyrics` route (lrclib → Genius → notFound)
25. `ManualLyricsInput` component for not-found fallback

Phase 7 — AI Furigana + Translation + Romaji Conversion

26. `src/lib/openrouter/romaji-to-japanese.ts` — convert romaji → kanji/kana via AI
27. `src/lib/openrouter/furigana.ts` (combined furigana+translation prompt)
28. `/api/ai/romaji-to-japanese` and `/api/ai/furigana` routes (batched 25 lines)
29. `RubyText`, `LyricsLine`, `LyricsDisplay` components
30. **Karaoke sync**: `LyricsDisplay` uses `progress_ms` from `useNowPlaying` to compute active LRC line → highlight + `scrollIntoView` smooth scroll
31. `useLyrics` + `useFurigana` hooks with non-JP / romaji / not-found state handling
32. Wire `LyricsPageClient` end-to-end

Phase 8 — Save-to-Dictionary from Lyrics

33. `PhraseSelector` (selectionchange listener + popover)
34. `SaveToDictionaryModal` (pre-filled context + AI enrich)

35. Test: play song → select phrase → save → verify in dictionary

Phase 9 — Flashcards

31. `FlashcardSetup` (filter by date/tags, mode selector)
32. `FlashcardCard` with CSS 3D flip (`transform-style: preserve-3d`)
33. `FlashcardSession` state machine: setup → reviewing → summary
34. `SessionSummary`

Phase 10 — Polish

35. Loading skeletons, error states, empty states
36. Mobile responsive pass
37. Test with second Supabase account to verify RLS isolation

---

Environment Variables Required

```
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=        # Server-only, never NEXT_PUBLIC_
OPENROUTER_API_KEY=
SPOTIFY_CLIENT_ID=
SPOTIFY_CLIENT_SECRET=
GENIUS_ACCESS_TOKEN=
NEXT_PUBLIC_APP_URL=              # e.g. http://localhost:3000
# lrclib.net requires no API key
```

Key Risks

Risk	Mitigation
lrclib.net coverage gaps	Fall back to Genius; if both fail, show manual paste UI — user can still get furigana+translation
Genius HTML structure changes	Use `data-lyrics-container="true"` selector; add fallback `div.lyrics`; set realistic User-Agent header
Romaji→kanji AI accuracy	Romaji→hiragana is deterministic; hiragana→kanji is AI-estimated. Mark converted lyrics visually so user knows it's AI-inferred, not original.
Karaoke sync drift	LRC timestamps from lrclib.net may be slightly off. Add a user-adjustable offset slider (±5s) as a nice-to-have.
OpenRouter returns malformed JSON	Strip markdown fences before parse; use `response_format: json_object`; retry once on failure
Spotify token refresh race (multiple tabs)	Check `expires_at > now + 5min` buffer; acceptable for personal multi-tab use
Free model furigana accuracy	AI is a learning aid, not authoritative — users can edit saved entries
Long lyrics exceed token limit	Batch processing: 25 lines per OpenRouter request, progressive rendering
Service role key exposure	Never use `NEXT_PUBLIC_` prefix; service role only in `auth/callback` and `spotify/callback` routes

---

Verification

1. **Auth**: Sign up with email, sign in with Google, session persists across page reload
2. **Dictionary**: Add a word manually → AI populates fields → save → appears in table → edit → delete → RLS: second user cannot see first user's words
3. **Spotify**: Connect → play a Japanese song → NowPlayingBanner updates → lyrics load with furigana and translations rendered
4. **Save from lyrics**: Select a phrase in lyrics view → modal opens pre-filled → save → appears in dictionary with song source
5. **Flashcards**: Create session with date filter → flip card → mark got-it/missed → view summary → retry missed
6. **Sorting**: Verify all 4 sort modes + search + tag filter work correctly